Spring Boot 3.X 개발 기본

inky4832@daum.net

2. Spring Boot 핵심 기능

- 의존성 주입과 IoC 컨테이너
- Configuration과 Profile 관리
- Spring Boot Actuator
- 예외 처리와 검증

2.1 의존성 주입과 IoC 컨테이너

IoC (Inversion of Control)

- 개념
  - 객체의 생성 및 의존성 관리를 개발자가 아닌 Spring 컨테이너가 대신 수행하는 방식
  - 개발자가 new로 직접 생성하지 않고 Spring이 대신 객체를 생성 및 관리
  - 이를 통해 코드의 결합도를 낮추고 유지보수와 테스트가 쉬워짐

- 기존 방식의 문제점
  - 클래스들간 강결합 (구현체 클래스에 직접 의존)
  - 코드 수정 및 유지보수 어려움

ex.

```
public class OrderService {
    private JpaOrderRepository orderRepository = new JpaOrderRepository();        // 직접 생성 (강결합)
   // private JdbcOrderRepository orderRepository = new JdbcOrderRepository();  // 직접 생성 (강결합)
}
```

2.1 의존성 주입과 IoC 컨테이너

IoC (Inversion of Control)

- 기존 방식의 문제점 해결
  - 객체생성과 주입은 Spring IoC 컨테이너가 담당
  - 개발자는 단지 연결 관계(interface)만 정의
  - 코드 변경없이 다른 구현체로 쉽게 교체 가능 (유연하고 테스트 용이)

```
ex.
public class JpaOrderRepository implements OrderRepository{}
public class JdbcOrderRepository implements OrderRepository{}

public class OrderService {

   private final OrderRepository orderRepository;

   // 외부에서 주입 받음 ( 약결합 )
   public OrderService(OrderRepository orderRepository) {
            this.orderRepository = orderRepository;
   }
}
```

2.1 의존성 주입과 IoC 컨테이너

기존 방식 vs IoC 방식

항목	기존방식 (직접 제어)	IoC 방식 (Spring 컨테이너 제어)
객체 생성	개발자가 직접 new 로 생성	컨테이너가 자동 생성 @Component, @Service, @Controller,@Repository 등
의존성 주입	직접 코드내에서 주입	생성자 또는 @Autowired 이용
관리 주체	개발자	Spring IoC 컨테이너
테스트 용이성	낮음 (강결합)	높음(느슨한 결합)
유지보수성	낮음	높음

2.1 의존성 주입과 IoC 컨테이너

IoC 컨테이너

- 개념
  - Spring 에서 IoC는 컨테이너(Container)를 통해 구현
  - IoC 방법으로 Bean을 관리한다는 의미에서 IoC 컨테이너라고 부름
  - 컨테이너 자체도 Bean으로서 BeanFactory와 ApplicationContext가 컨테이너가 구현해야 될 기본 인터페이스임

- 주요 역할

기능	설명
Bean 생성	설정정보(@Configuration, @ComponentScan) 기반으로 Bean 자동생성
의존성 주입	필요한 객체를 자동으로 연결 (생성자 또는 @Autowired 이용)
생명주기 관리	Bean 의 생성 -> 초기화 -> 사용 -> 소멸까지의 전 과정을 관리
설정 및 환경 관리	외부 설정 파일 (application.yml)과 연동

2.1 의존성 주입과 IoC 컨테이너

IoC 컨테이너 주요 인터페이스

인터페이스	설명
BeanFactory	가장 기본적인 IoC 컨테이너
ApplicationContext	BeanFactory를 확장한 고급 컨테이너 메시지소스, 이벤트 발행, 환경설정 관리 등 지원
WebApplicationContext	웹 어플리케이션용 IoC 컨테이너 Spring MVC와 연동됨

2.1 의존성 주입과 IoC 컨테이너

IoC 관련 주요 특징

특징	설명
제어의 역전	객체의 생성 및 관리를 IoC 컨테이너가 담당
의존성 주입(DI)	객체 간의 관계를 자동으로 연결
느슨한 결합	코드 간 의존성이 낮아 유지보수 용이
확장성 향상	설정 변경만으로 쉽게 구현체 교체 가능
테스트 용이성	Mock 객체 활용하여 단위 테스트 가능
재사용성 향상	Bean 재활용 가능 (Singleton 기본)

2.1 의존성 주입과 IoC 컨테이너

POJO (Plain Old Java Object)

- 개념
  - 특정 프레임워크 규약/환경에 종속되지 않는 순수한 객체 의미
  - Spring 은 POJO 중심으로 Bean들을 결합해 확장성/테스트 용이성 보장

- 특징
  - Spring 프레임워크 핵심
  - 특정 규약에 종속되지 않음
  - 특정 환경에 종속되지 않음
  - 객체 지향적인 원리에 충실

2.1 의존성 주입과 IoC 컨테이너

의존성 주입 (Dependency Injection, DI)

- 개념
  - 객체가 의존하는 다른 객체를 직접 생성하지 않고 외부에서 주입 받는 방식
  - 핵심원리는 IoC 컨테이너가 객체 간의 관계를 자동으로 연결해줌

- 특징
  - IoC의 구체적인 구현 방법
  - 객체 간 결합도(Coupling)를 낮춤
  - 유연성과 재사용성 향상

2.1 의존성 주입과 IoC 컨테이너

의존성 주입 3가지 방식

주입방식	설명	장점	단점
생성자 주입 (Constructor Injection)	생성자를 통해서 의존 객체를 전달	불변성 보장, 필수 의존성 명확	순환 참조 시 이슈 가능
Setter 주입 (Setter Injection)	Setter 메서드를 통해서 주입	선택적 의존성 처리 가능	객체가 완전하지 않은 상태로 사용될 수 있음.
필드 주입 (Field Injection)	@Autowired로 필드 직접 주입	코드 간결	테스트/DI 프레임워크 외부 사용 어려움

2.1 의존성 주입과 IoC 컨테이너

생성자 주입 (Constructor Injection) 예시

```
@Service
public class OrderService {
    private final OrderRepository orderRepository;

    // 생성자 기반 주입
public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    public void processOrder() {
        orderRepository.saveOrder();
    }
}
```

2.1 의존성 주입과 IoC 컨테이너

Setter 주입 (Setter Injection) 예시

```
@Service
public class PaymentService {
    private PaymentRepository paymentRepository;

    // Setter 기반 주입
    @Autowired
    public void setPaymentRepository(PaymentRepository paymentRepository) {
        this.paymentRepository = paymentRepository;
    }

    public void pay() {
        paymentRepository.savePayment();
    }
}
```

2.1 의존성 주입과 IoC 컨테이너

필드 주입 (Field Injection) 예시

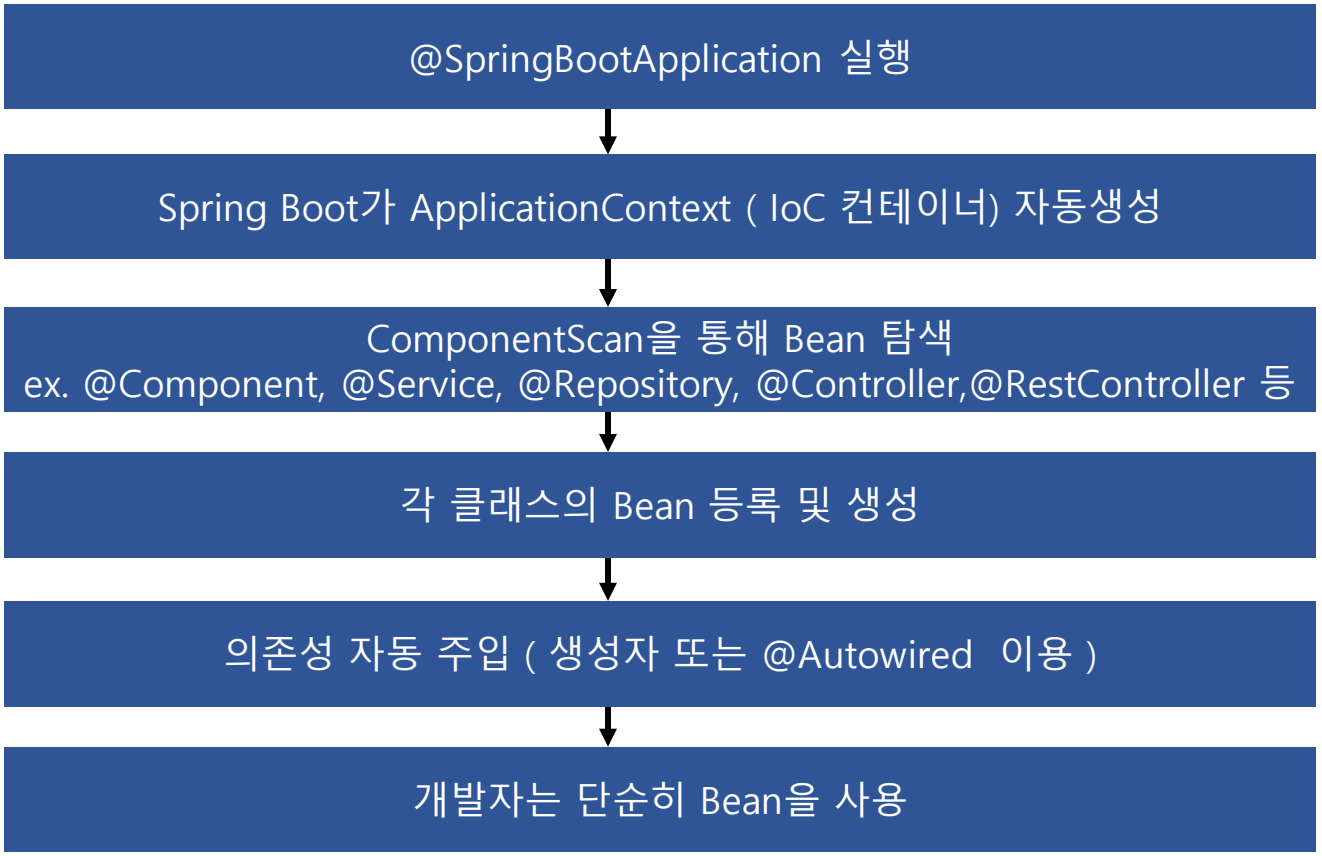
```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public void printUser() {
        System.out.println(userRepository.findUser());
    }
}
```

2.1 의존성 주입과 IoC 컨테이너

DI 적용 흐름



2.1 의존성 주입과 IoC 컨테이너

@Autowired 동작 원리

- 개념
  - IoC 컨테이너에 등록된 Bean 중에서 **타입(Type)**이 일치하는 객체를 자동으로 주입함
  - 만약 타입(Type)이 일치하는 객체가 여러 개인 경우 필드명이 일치하는 객체가 주입됨.
- 예외 사항
  - 타입(Type)이 일치하는 Bean이 둘 이상인 경우 예외 발생 가능성 있음
    ex. 필드명이 일치하는 객체가 없는 경우
- 예외 사항 해결
  - @ Qualifier 또는 @Primary 이용해서 명시적으로 주입할 객체 지정

2.1 의존성 주입과 IoC 컨테이너

@Qualifier 예시 (특정 Bean 지정)

```
@Repository("mysqlRepository")
public class MySqlOrderRepository implements OrderRepository {}

@Repository("mongoRepository")
public class MongoOrderRepository implements OrderRepository {}

@Service
public class OrderService {

    @Autowired
    @Qualifier("mongoRepository")
    private OrderRepository repository;
}
```

2.1 의존성 주입과 IoC 컨테이너

@Primary 예시 (기본 Bean 우선 지정)

```
@Repository
@Primary
public class DefaultOrderRepository implements OrderRepository {}

@Repository("mysqlRepository")
public class MySqlOrderRepository implements OrderRepository {}

@Repository("mongoRepository")
public class MongoOrderRepository implements OrderRepository {}

@Service
public class OrderService {

   @Autowired
   private OrderRepository repository;
}
```

2.1 의존성 주입과 IoC 컨테이너

Bean 생명주기 콜백 메서드

- 개념

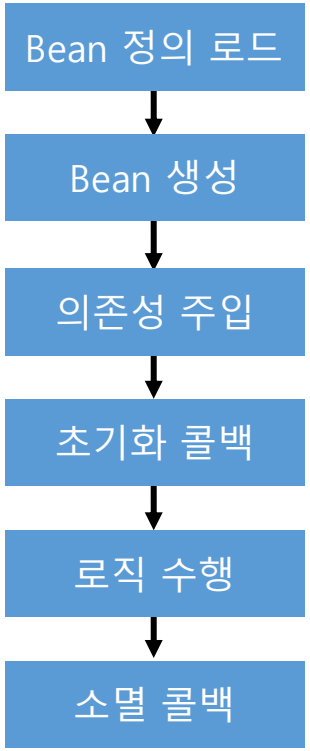
  IoC 컨테이너에 저장되는 Bean 객체는 생성/초기화/소멸의 생명주기를 가짐
  이때 특정 상태에서 자동으로 호출되는 콜백 메서드를 사용하여 빈 객체의 상태를 정교하게 제어 가능

- 방법

어노테이션	설명
@PostConstruct	Bean 생성 및 의존성 주입이 끝난 직후 1회 호출 컨테이너가 해당 메서드를 실행한 후 Bean이 준비상태가 됨
@PreDestroy	어플리케이션 컨텍스트가 종료 될 때 cleanup 용으로 1회 호출 연결 자원 정리, 스레드 풀 종료, 버퍼 flush 등 수행

2.1 의존성 주입과 IoC 컨테이너

Bean 생명주기 흐름



@Component,@Service등으로 Bean 정보 읽기

인스턴스 생성

생성자 또는 @Autowired 이용하여 필요한 Bean 주입

@PostConstruct 등 초기 설정 수행

실제 서비스 로직 수행

@PreDestroy 호출, 리소스 정리 수행

2.2 Configuration과 Profile 관리

Configuration

- 개념
  - 어플리케이션 실행 시 필요한 각종 환경 정보 (서버 포트, DB, API Key, 로그레벨 등)를 코드가 아닌 외부 파일 또는 환경변수로 분리해 관리하는 것을 의미
  - 즉, 코드를 수정하지 않고 환경에 따라 설정만 바꿔 어플리케이션 실행이 가능하도록 지원

- 구성요소 예시

설정 항목	예시
서버 포트	server.port=8080
DB 연결 정보	spring.datasource.url=jdbc:mariadb://localhost:3306/test
로그 레벨	logging.level.org.springframework=DEBUG
외부 API Key	weather.api.key=1234567890abcdef

2.2 Configuration과 Profile 관리

Configuration 문제점 및 해결

- 기존 방식 문제점
  - 설정이 코드에 하드코딩 되어 환경이 바뀌면 재컴파일 필요
  - 테스트 환경 및 운영 환경이 달라 오류 발생 가능성 증가
  - 민감한 정보(DB 비밀번호, API Key 등)가 코드에 노출

- 문제점 해결
  - 설정 정보를 코드 밖 (application.yml, 환경 변수 등)으로 추출
  - 실행 시점에 필요한 설정을 주입 받아서 실행
  - 재배포 없이 실행 환경 전환 가능

2.2 Configuration과 Profile 관리

Configuration 특징

- 유지보수성 향상
  - 설정 변경 시 코드 수정 없이 환경 전환 가능
- 보안성 확보
  - 비밀번호, API Key 등 민감정보를 코드 외부로 분리
- CI/CD 자동화
  - 환경 변수 기반 자동 빌드 및 배포 지원
- 가시성 향상
  - 설정 파일만 봐도 시스템 구조 파악 가능

2.2 Configuration과 Profile 관리

Configuration 구조

SpringApplication
- **Environment**
  - **PropertySources (리스트형)**
    - CommandLinePropertySource
    - SystemProperties
    - SystemEnvironment
    - ConfigData (application.yml)
    - DefaultProperties
  - **Profiles**
    - ActiveProfiles (spring.profiles.active)
    - DefaultProfiles (spring.profiles.default)

2.2 Configuration과 Profile 관리

Environment

- 개념

  Spring Boot 어플리케이션이 외부 설정값 (application.yml, 환경변수 등)을 읽어와
  현재 실행 환경(dev/test/prod)을 구분하며 관리하는 중앙 제어 객체
  어플리케이션이 시작될 때 자동 생성되어 모든 Bean, 설정파일, AutoConfiguration의 값 바인딩 근거가 됨.

- 핵심 기능

기능	설명
Property 관리	다양한 설정 소스 (yml, 명령줄)의 값을 통합 저장
Profile 관리	실행 환경 (개발, 테스트, 운영 등) 구분 및 활성화
값 조회 제공	@Value, @ConfigurationProperties 등에서 사용

2.2 Configuration과 Profile 관리

Environment 필요성

- 외부 환경별 유연한 설정관리

  개발(dev), 운영(prod) 환경별로 DB, 서버포트, 로깅 등 설정을 분리

- 코드와 설정 분리 (Configuration Separation)

  설정값이 코드에 고정되지 않도록 분리

  유지보수 용이

- 다양한 설정 소스 통합 관리

  명령줄 옵션, 시스템 변수, yml 파일 등 여러 접근을 하나로 관리

- 자동 구성(AutoConfiguration) 기반

  Spring Boot의 자동 설정은 Environment에서 값을 가져와 동작

2.2 Configuration과 Profile 관리

PropertySource

- 개념
  - 설정값(key/value)을 보관하는 하나의 저장소 단위
  - **Environment 내에 리스트로 순서대로 쌓고 Top-Down 방식으로 조회**

- 기존 방식 문제점
  - 개발/운영 환경마다 DB, 포트, 로그 레벨이 다름
  - 설정 방식이 다양 ex. 명령줄, OS 환경변수, yml, 코드 기본값
  - 같은 key 존재 시 어느 값이 적용되는지 혼란

- 문제점 해결
  - ProperySource를 리스트로 쌓고 위에서부터 값을 조회 (우선순위 규칙)
  - Environment 가 최종 1개 값으로 정리해 반환

2.2 Configuration과 Profile 관리

PropertySource 우선 순위

우선순위	이름	출처	예시	비고
1	commandLineArgs	명령줄 인자	--server.port=9090	최상위, java –jar 옵션
2	systemProperties	JVM 인자	-Dserver.port=8085	JVM 옵션
3	systemEnvironment	OS 환경변수	SERVER_PORT=8083	CI/CD, Docker 사용
4	applicationConfig	yml/properties	server.port=8080	파일 기반 구성
5	defaultProperties	코드 주입	setDefaultProperties	최하위

2.2 Configuration과 Profile 관리

값 주입(Value Injection)

- 개념
  - application.yml 파일에 정의된 설정 값을 자바 코드(Bean)내부로 전달하여 사용하는 방법

- 구현

구분	주용도	어노테이션	특징
단일값	간단한 값 ex. 문자열,숫자	@Value	빠르고 간단
여러값	구조적/복합 설정	@ConfigurationProperties	타입 안정, 유지보수 탁월

2.2 Configuration과 Profile 관리

@Value

- 개념
  - 한두 개의 단일 값을 빠르게 주입할 때 사용
  - application.yml의 특정 키를 읽어서 변수에 직접 할당
- 문법
  - **@Value(“${key값:기본값}”)**

2.2 Configuration과 Profile 관리

@Value 장단점

- 장점
  - 간단하고 빠름
  - 별도의 클래스 정의 필요 없음

- 단점
  - 값이 여러 클래스에 흩어짐 (스파게티 구조 위험)
  - 타입 안정성 및 유효성 검증 불가
  - 대규모 프로젝트에서는 유지보수 어려움

2.2 Configuration과 Profile 관리

@ConfigurationProperties

- 개념
  - 관련된 여러 설정 값을 하나의 객체(Bean)로 묶어 주입
  - 계층형 구조인 YAML과 자연스럽게 매핑됨
  - 타입 검증과 자동완성 지원
  - Spring Boot에서 권장하는 표준적인 설정 주입 방식

- 특징
  - 구조적 바인딩
  - 타입 안정성(Type-safe)
  - 자동 변환 지원
  - 검증 (Validation) 지원
  - IDE 자동완성 지원
  - 스캔 가능

2.2 Configuration과 Profile 관리

@ConfigurationProperties

- 구현 방법
  - A. 스캔 활성화

    ex. **@ConfigurationPropertiesScan**

    ```
    @SpringBootApplication
    public class DemoApplication{}
    ```

  - B. 프로퍼티 클래스 정의

    ex. **@ConfigurationProperties (prefix="app")**

    ```
    public record AppProps {}
    ```

  - C. 주입 및 사용

    ex. **@Autowired**

    ```
    AppProps appProps;
    ```

2.2 Configuration과 Profile 관리

비교 요약

비교 항목	@Value	@ConfigurationProperties
주입 대상	단일 값	여러 값(객체 단위)
타입 변환	문자열 기반 수동 변환	자동 변환 ex. Duration, DataSize, Enum 등
검증 기능	불가	@Validated 로 검증 가능
IDE 자동완성	불가	가능 (prefix 기반)
유지 보수	낮음	높음
사용 예	간단한 값	구조적/프로젝트 전역 설정

2.2 Configuration과 Profile 관리

검증(Validation)

- 개념
  - 입력값이나 설정값이 올바른지 확인하는 과정
  - @ConfigurationProperties 등의 데이터가 규칙에 맞는지 자동으로 검사하는 기능 제공
- 의존성 추가
  - implementation 'org.springframework.boot:spring-boot-starter-validation'
- 포함 라이브러리
  - Jakarta.validation-api
    검증 표준 인터페이스 제공 ex. @NotBlank, @Max, @Min 등
  - hibernate-validator
    실제 검증 로직 구현체

2.2 Configuration과 Profile 관리

제약(Constraint) 어노테이션

어노테이션	설명	예시
@NotNull	null 금지	@NotNull String host
@NotBlank	공백 문자열 금지	@NotBlank String password
@Min, @Max	숫자 범위 제한	@Min(1) @Max(10)
@Size	문자열/리스트 크기 제한	@Size (min=3, max=10)
@Email	이메일 형식	@Email String email
@Pattern	정규식 매칭	@Pattern(regexp="^[a-z]"

2.2 Configuration과 Profile 관리

Validation 주요 특징

항목	설명
타입 안정성	잘못된 타입 자동 감지
유효성 제약 설정 가능	@NotBlank, @Min, @Max 등 조건 지정
부팅 시점 검증	설정값 오류는 앱 시작 전에 실패
자동 오류 메시지 출력	어떤 값이 잘못됐는지 명확하게 표시
@Validated 필수	검증 활성화 방법으로 클래스에 지정
검증 실패 시 부팅 중단	어플리케이션이 실행되지 않음

2.2 Configuration과 Profile 관리

Profile

- 개념
  - dev, test, prod 등 환경별로 다른 설정을 자동으로 적용하는 기능
  - 즉 현재 실행중인 프로파일이 무엇인지에 따라서 다른 DB 설정, 다른 로깅 설정,
  - 다른 서버 포트 등 서로 다른 설정이 가능

- 주요 특징

구분	설명
환경 분리	개발(dev), 테스트(test), 운영(prod) 환경에 따라 서로 다른 설정 파일을 분리 가능
유연한 실행	실행시점에 --spring.profiles.active 값만 변경해서 손쉽게 환경 전환이 가능
자동 병합	공통 설정(application.yml)과 환경별 설정(application-{프로파일}.yml)을 **자동 병합**
코드 수준 분리	@Profile 어노테이션을 사용하여 특정 환경에서만 Bean 등록 가능

2.2 Configuration과 Profile 관리

Profile

- 구성 파일
  문법: application-{프로파일}.yml

  ex.
  공통: application.yml
  개발: application-**dev**.yml
  테스트: application-**test**.yml
  운영: application-**prod**.yml

- 활성화

  java -jar myapp.jar **--spring.profiles.active=prod**
  ./gradlew bootRun **--args='--spring.profiles.active=prod'**

2.2 Configuration과 Profile 관리

실습 프로젝트 구조 (Profile 기본)

```
src/main/
    java/
        com/
            example/
                demo/
                    DemoApplication.java            # @SpringBootApplication

    resources/
        application.yml                             # 공통
        application-dev.yml                         # 개발 프로파일
        application-prod.yml                        # 운영 프로파일
```

수고하셨습니다.